

LECTURE 4

Neural networks

Foundations of Artificial Intelligence

SEMTM0051 · 2026/27

Lecturer: Dr Roberto Colomboni



In this lecture...

We will build neural networks using the mathematics we already know.

Affine layers and activation functions

Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

What we fitted before

In linear regression, we chose the feature functions in advance.

$$f(x) = b + w_1x + w_2x^2$$

Fixed feature functions

$$\phi_1(x) = x, \quad \phi_2(x) = x^2$$

Adjustable coefficients

$$b, \quad w_1, \quad w_2$$

Can the data also help us choose the feature functions?

Learning the feature functions

A neural network places adjustable parameters inside the feature functions.

$$f(\mathbf{x}) = b + \sum_{j=1}^m w_j \phi_j(\mathbf{x})$$

**Parameters inside the
functions**

$$\phi_1, \dots, \phi_m$$

**Parameters combining their
values**

$$b, w_1, \dots, w_m$$

First, let us try affine feature functions with adjustable coefficients.

A first idea: affine feature functions

A first idea is to make each feature an **affine** function of the input.

$$\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^{1 \times d}, \quad \mathbf{W} = \begin{pmatrix} W_{11} & \cdots & W_{1m} \\ \vdots & \ddots & \vdots \\ W_{d1} & \cdots & W_{dm} \end{pmatrix} \in \mathbb{R}^{d \times m}, \quad \mathbf{b} = (b_1, \dots, b_m) \in \mathbb{R}^{1 \times m}$$

$$\phi_j(\mathbf{x}) = \sum_{k=1}^d x_k W_{kj} + b_j, \quad j = 1, \dots, m$$

$$\boldsymbol{\phi}(\mathbf{x}) = (\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x})) = \mathbf{x}\mathbf{W} + \mathbf{b} \in \mathbb{R}^{1 \times m}$$

The weights $\mathbf{W}$ and biases $\mathbf{b}$ now determine the feature functions.

An affine feature and an affine output

The **two affine** stages are **equivalent** to a **single affine** predictor.

Feature function

$$\phi(x) = w_1x + b_1$$

$$w_1 = 2, \quad b_1 = 1$$

Affine output

$$f(x) = w_2\phi(x) + b_2$$

$$w_2 = 3, \quad b_2 = -4$$

$$f(x) = 3(2x + 1) - 4 = 6x - 1$$

The same predictor directly from x , with no intermediate feature:

$$f(x) = \tilde{w}x + \tilde{b}, \quad \tilde{w} = 6, \quad \tilde{b} = -1$$

Affine features still give an affine predictor

The same limitation holds for any number of affine features.

$$\boldsymbol{\phi}(\mathbf{x}) = \mathbf{x}\mathbf{W} + \mathbf{b}$$

Output weights and bias: $\mathbf{w}_{\text{out}} \in \mathbb{R}^{m \times 1}, \quad b_{\text{out}} \in \mathbb{R}$

$$f(\mathbf{x}) = \boldsymbol{\phi}(\mathbf{x})\mathbf{w}_{\text{out}} + b_{\text{out}}$$

$$f(\mathbf{x}) = \mathbf{x} \underbrace{\mathbf{W}\mathbf{w}_{\text{out}}}_{=:\tilde{\mathbf{w}}} + \underbrace{\mathbf{b}\mathbf{w}_{\text{out}} + b_{\text{out}}}_{=:\tilde{b}} = \mathbf{x}\tilde{\mathbf{w}} + \tilde{b}$$

f is still an affine function with equivalent weight column and bias:

$$\tilde{\mathbf{w}} := \mathbf{W}\mathbf{w}_{\text{out}} \in \mathbb{R}^{d \times 1}, \quad \tilde{b} := \mathbf{b}\mathbf{w}_{\text{out}} + b_{\text{out}} \in \mathbb{R}$$

Repeating the argument shows that any number of affine stages remains affine.

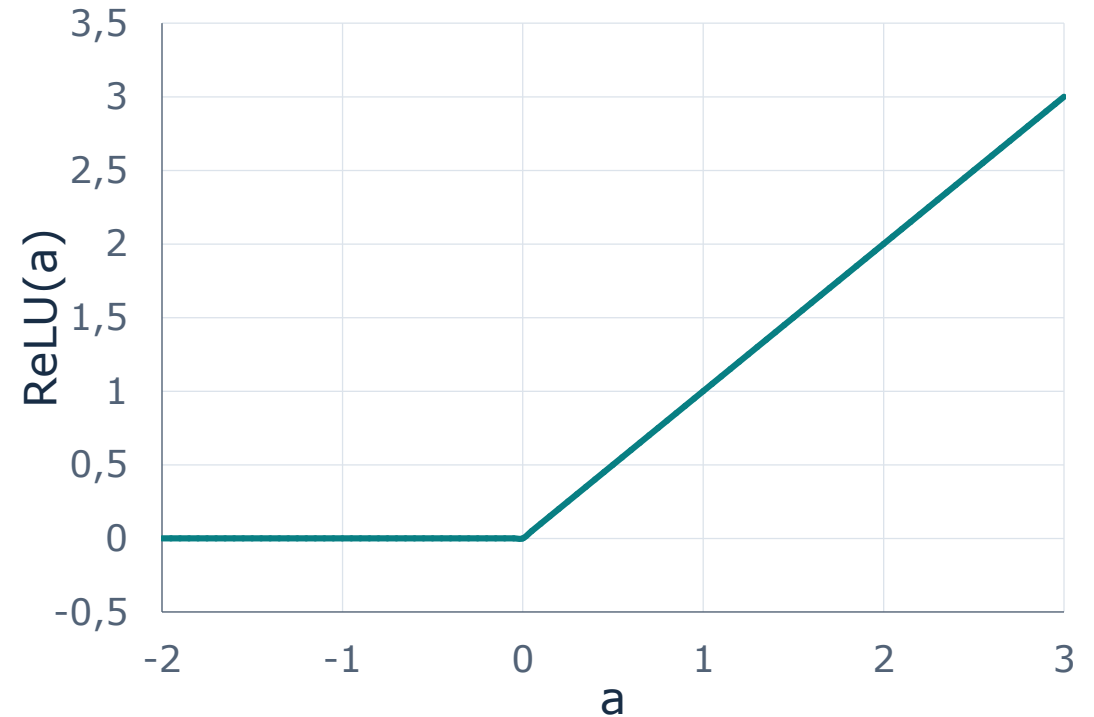
We need a **nonlinear operation **between** the affine stages.**

The ReLU activation function

We now apply a **nonlinear** function to each affine value.
We use **ReLU** (short for Rectified Linear Unit).

$$\text{ReLU}(a) = \max\{0, a\}$$

$$= \begin{cases} 0 & a \leq 0 \\ a & a > 0 \end{cases}$$



Each feature combines an affine transformation with ReLU.

In this lecture...

We will build neural networks using the mathematics we already know.

Affine layers and activation functions

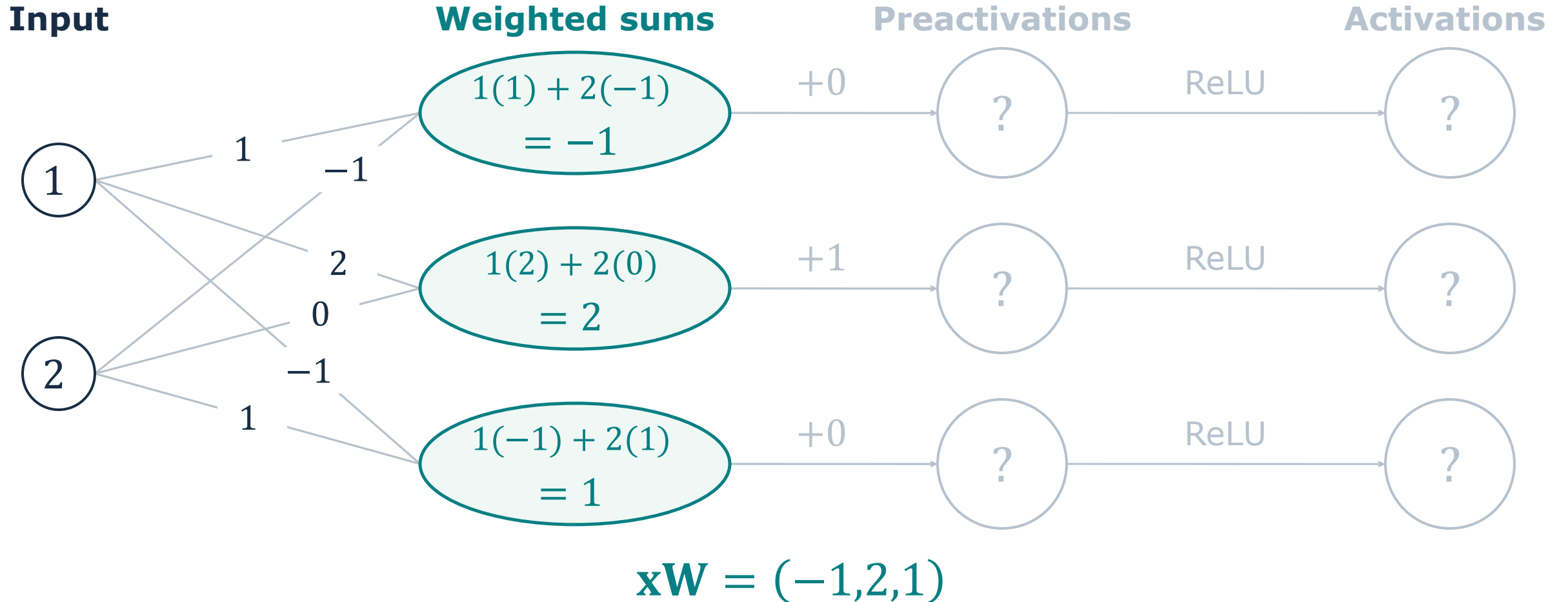
Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

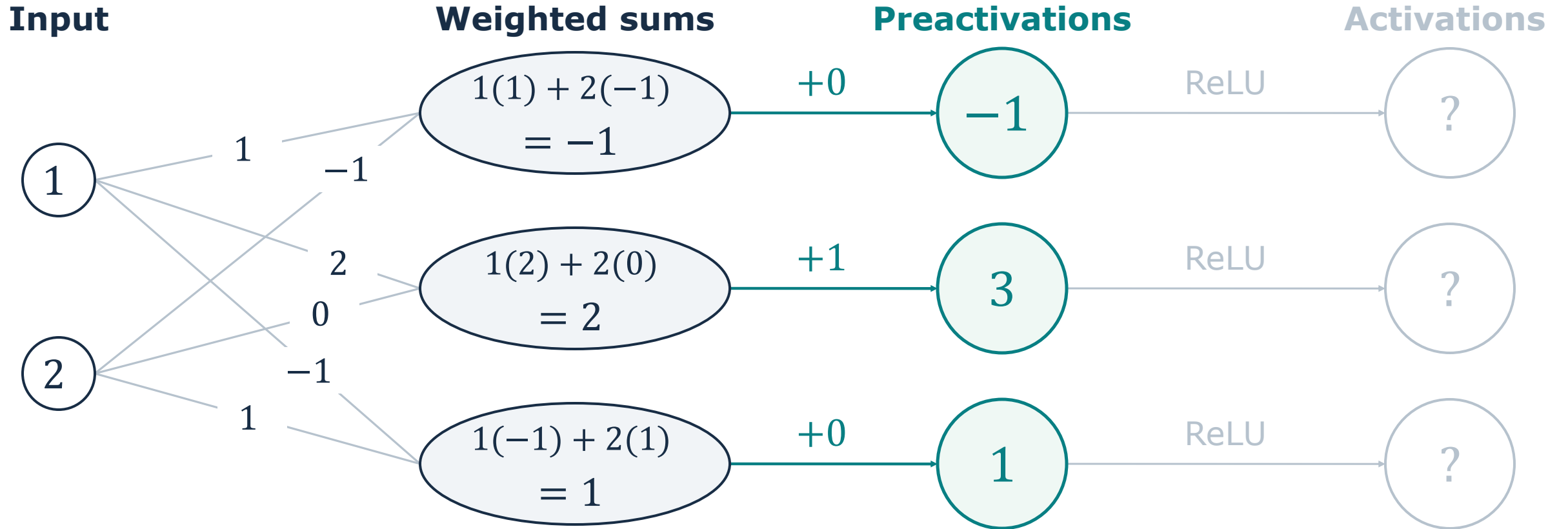
A first example

$$\mathbf{x} = (1, 2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0, 1, 0)$$



A first example

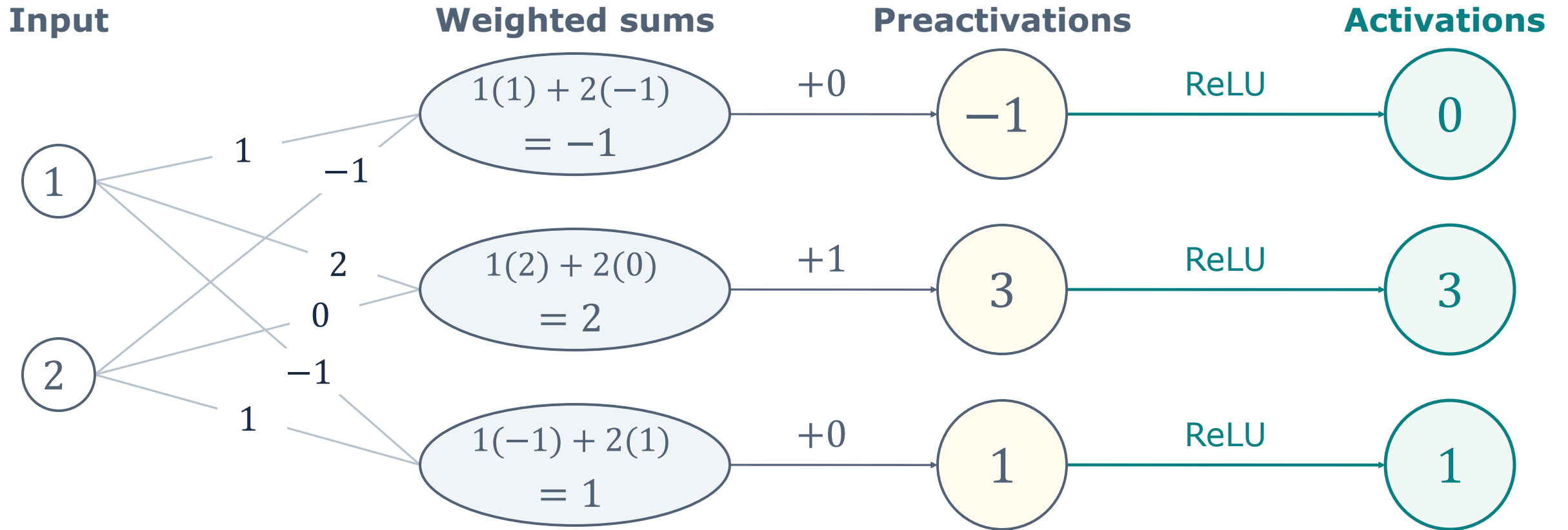
$$\mathbf{x} = (1,2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0,1,0)$$



$$(a_1((1,2)), a_2((1,2)), a_3((1,2))) = (-1, 3, 1)$$

A first example

$$\mathbf{x} = (1,2), \quad \mathbf{W} = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \mathbf{b} = (0,1,0)$$



$$\left(\phi_1((1,2)), \phi_2((1,2)), \phi_3((1,2)) \right) = (0,3,1)$$

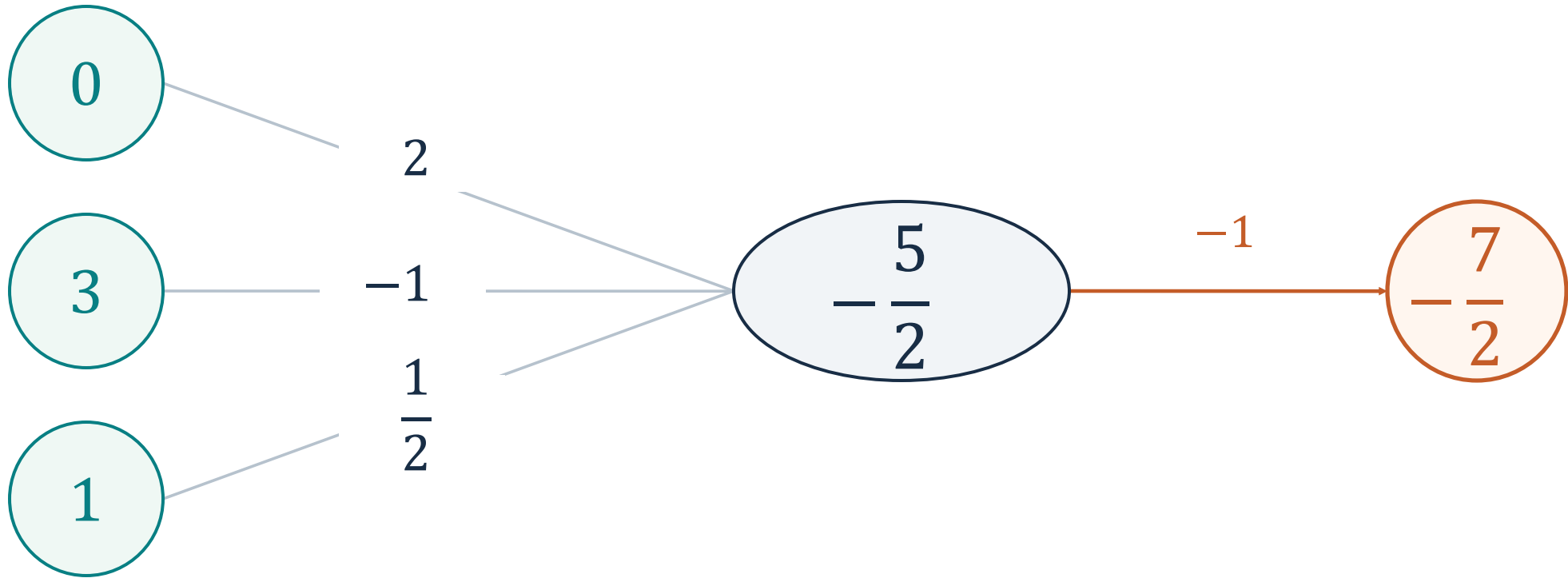
A first example

$$\mathbf{W}_{\text{out}} = \left(2, -1, \frac{1}{2} \right)^{\top}, \quad b_{\text{out}} = -1$$

Hidden unit

Weighted sum

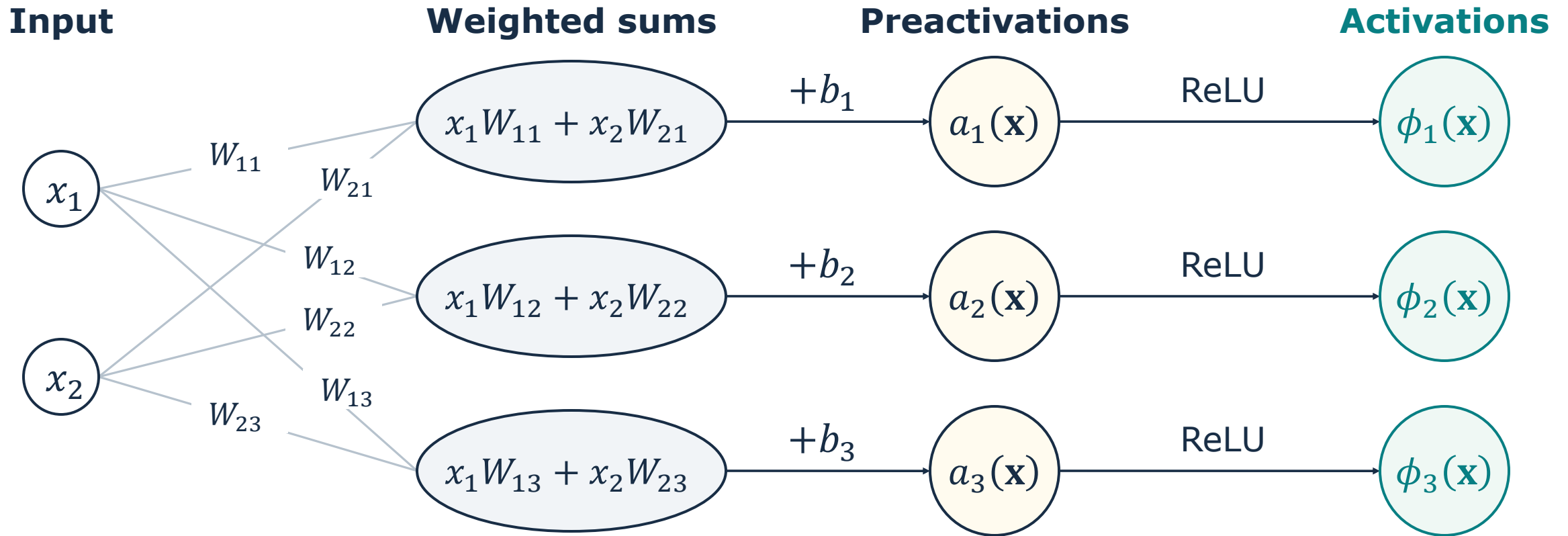
Output



$$f((1,2)) = 0 \cdot (2) + 3 \cdot (-1) + 1 \cdot \left(\frac{1}{2} \right) - 1 = -\frac{7}{2}$$

A hidden layer

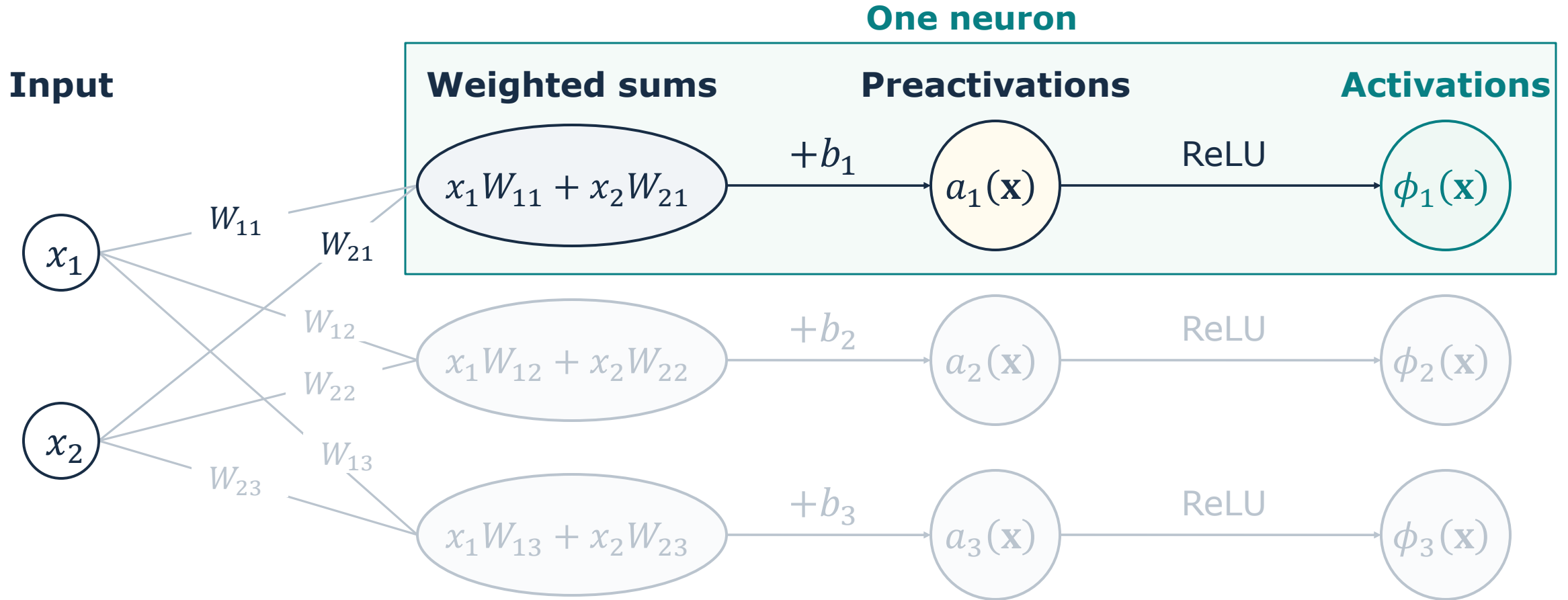
A **hidden unit** (or **neuron**) computes one feature value. It forms a weighted sum, adds a bias and applies an **activation function**. Here we use ReLU.



Each horizontal row follows one hidden unit (or neuron).

One hidden neuron

The highlighted row shows the complete calculation of one hidden neuron.



**The weighted sum, preactivation and activation belong to the same neuron.
This hidden layer contains three neurons.**

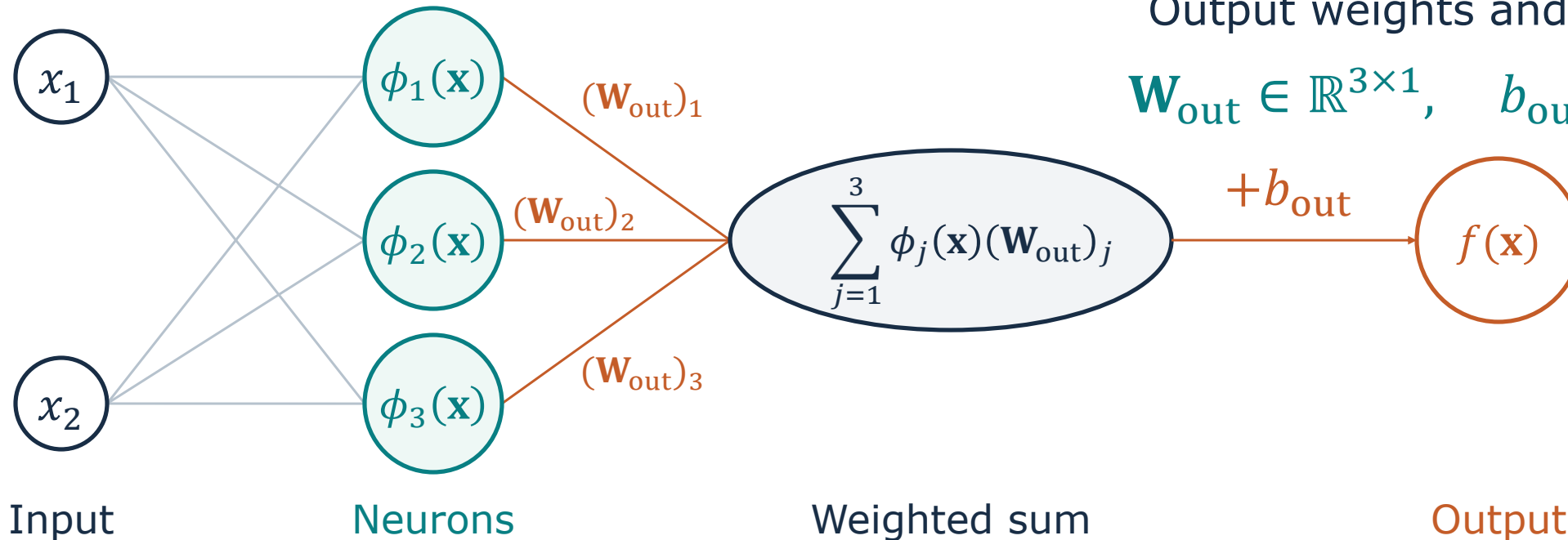
The output layer

Once we have the hidden activations, we form a weighted sum and add an output bias.

$$f(\mathbf{x}) = \sum_{j=1}^3 \phi_j(\mathbf{x})(\mathbf{W}_{\text{out}})_j + b_{\text{out}} = (\phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \phi_3(\mathbf{x}))\mathbf{W}_{\text{out}} + b_{\text{out}}$$

Output weights and bias:

$$\mathbf{W}_{\text{out}} \in \mathbb{R}^{3 \times 1}, \quad b_{\text{out}} \in \mathbb{R}$$



Each hidden circle shows the activation produced by one neuron.

A Neural Network with one hidden layer

A hidden layer groups units receiving the same input row.
Width m is the number of units. Input and output dimensions are d and q .

$$\mathbf{W} \in \mathbb{R}^{d \times m}, \quad \mathbf{b} \in \mathbb{R}^{1 \times m}, \quad \mathbf{W}_{\text{out}} \in \mathbb{R}^{m \times q}, \quad \mathbf{b}_{\text{out}} \in \mathbb{R}^{1 \times q}$$

Preactivations: $\mathbf{a}(\mathbf{x}) := (a_1(\mathbf{x}), \dots, a_m(\mathbf{x})) = \mathbf{x}\mathbf{W} + \mathbf{b}$

Activations: $\boldsymbol{\phi}(\mathbf{x}) := (\phi_1(\mathbf{x}), \dots, \phi_m(\mathbf{x})) = \text{ReLU}(\mathbf{a}(\mathbf{x}))$

where $\text{ReLU}(\mathbf{a}) := (\text{ReLU}(a_1), \dots, \text{ReLU}(a_m))$

Output: $\mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}) := \boldsymbol{\phi}(\mathbf{x})\mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}}$

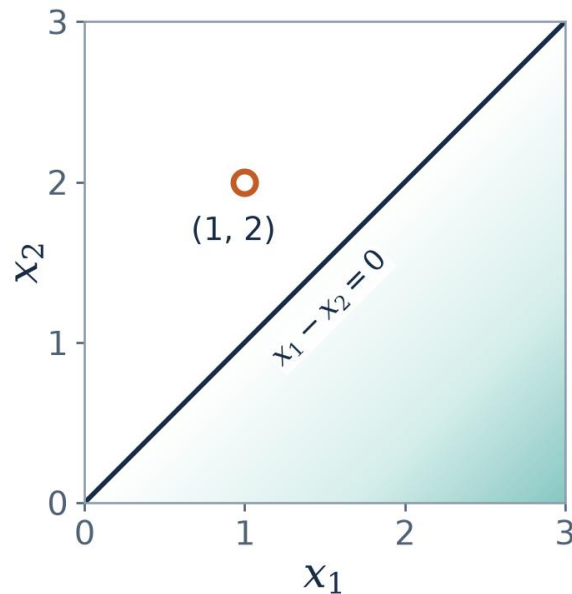
The column $\boldsymbol{\theta}$ collects all adjustable weights and biases, in a fixed order.

Learning changes the feature functions

An illustrative change to the first neuron's weights and bias.

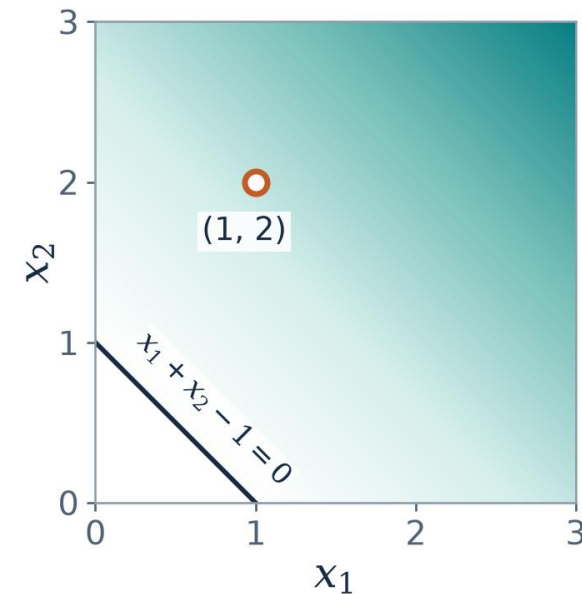
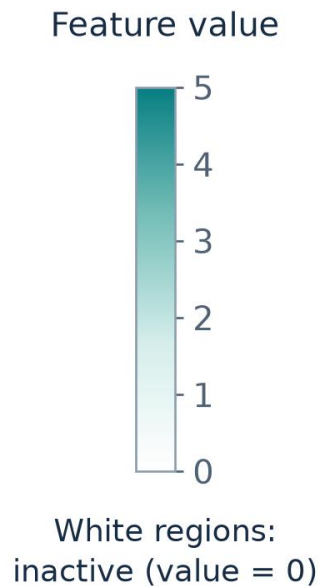
Original parameters

$$\phi_1(\mathbf{x}) = \text{ReLU}(x_1 - x_2)$$
$$(W_{11}, W_{21}, b_1) = (1, -1, 0)$$



Changed parameters

$$\tilde{\phi}_1(\mathbf{x}) = \text{ReLU}(x_1 + x_2 - 1)$$
$$(W_{11}, W_{21}, b_1) = (1, 1, -1)$$



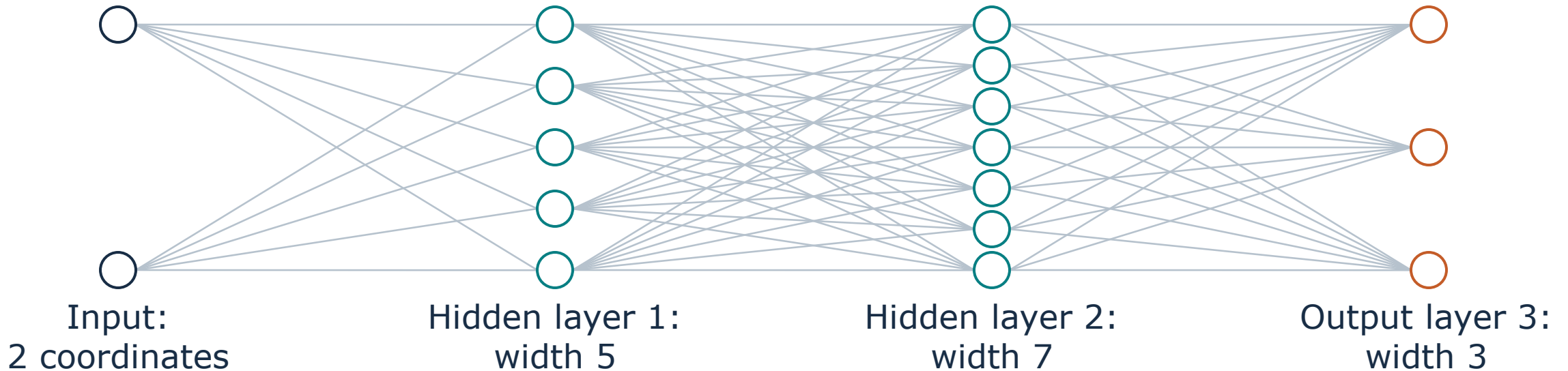
$$\mathbf{x} = (1, 2): \quad \phi_1(\mathbf{x}) = 0 \quad \rightarrow \quad \tilde{\phi}_1(\mathbf{x}) = 2$$

The input stays the same, but the feature value changes.

Depth, width and architecture

Width: the number of neurons in a layer.

Depth: the number of layers, counting the output but excluding the input.



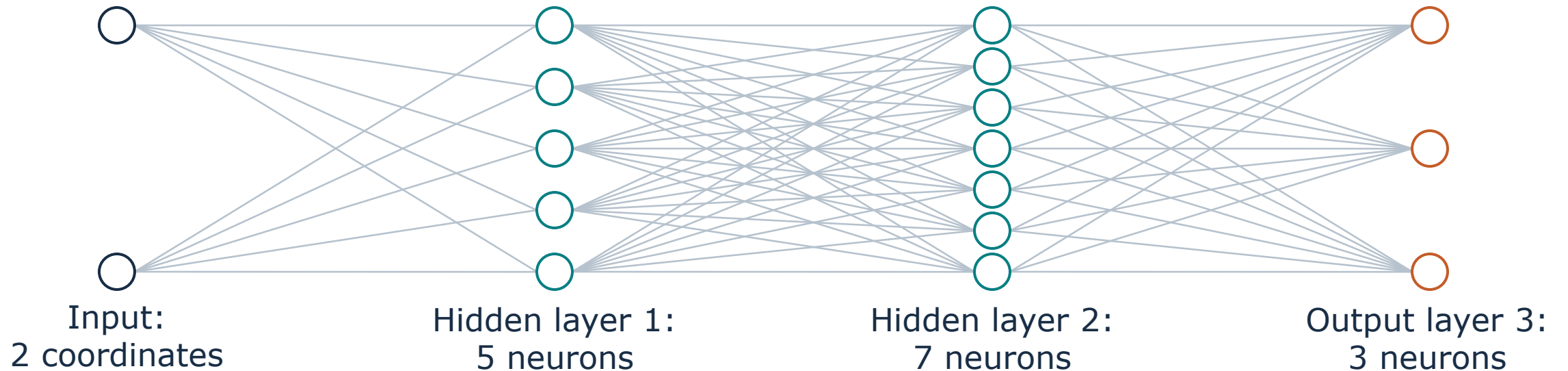
Depth = 3: two hidden layers and one output layer.

For fully connected networks with ReLU hidden layers and an affine output, the input dimension and layer widths specify the **architecture**: $2 \rightarrow 5 \rightarrow 7 \rightarrow 3$.

The architecture determines the shapes of the weight matrices and biases.
Training adjusts their values.

More hidden layers

We now index layers by r and units by j .
Depth L includes the output layer. Here $L = 3$.



$$\phi_{r,j}(\mathbf{x}) := [\boldsymbol{\Phi}_r(\mathbf{x})]_j$$

$$\boldsymbol{\Phi}_0(\mathbf{x}) = \mathbf{x}, \quad \boldsymbol{\Phi}_r(\mathbf{x}) = \text{ReLU}(\boldsymbol{\Phi}_{r-1}(\mathbf{x})\mathbf{W}_r + \mathbf{b}_r)$$

$$r = 1, \dots, L - 1, \quad \mathbf{f}_{\theta}(\mathbf{x}) = \boldsymbol{\Phi}_{L-1}(\mathbf{x})\mathbf{W}_L + \mathbf{b}_L$$

Dimensions and parameters

For the **architecture** $2 \rightarrow 5 \rightarrow 7 \rightarrow 3$
each layer has its own weights and biases.

Layer	Weight matrix	Bias row	Parameters
1	2×5	1×5	$10 + 5 = 15$
2	5×7	1×7	$35 + 7 = 42$
3	7×3	1×3	$21 + 3 = 24$

$$15 + 42 + 24 = 81$$

The parameter column θ has 81 entries.

Regression and binary classification

The same scalar network output can play different roles.

Regression

$$\hat{y} = f_{\theta}(\mathbf{x})$$

$$\ell_{\text{sq}}(\hat{y}, y) = (\hat{y} - y)^2$$

Use the squared loss.

Binary classification

$$p_{\theta}(\mathbf{x}) = \sigma(f_{\theta}(\mathbf{x}))$$

$$h_{p_{\theta}}(\mathbf{x}) = \begin{cases} 1 & p_{\theta}(\mathbf{x}) > 1/2 \\ 0 & p_{\theta}(\mathbf{x}) \leq 1/2 \end{cases}$$

Use the logistic loss.

For each task, neural networks use the same loss as the corresponding linear model.

Multiclass classification

With K classes, the network outputs a row of K real scores.

$$\mathbf{f}_{\theta}(\mathbf{x}) = \left(f_{\theta,1}(\mathbf{x}), \dots, f_{\theta,K}(\mathbf{x}) \right)$$

$$\mathbf{p}_{\theta}(\mathbf{x}) = \text{Softmax}(\mathbf{f}_{\theta}(\mathbf{x})), \quad p_{\theta,k}(\mathbf{x}) = \frac{e^{f_{\theta,k}(\mathbf{x})}}{\sum_{r=1}^K e^{f_{\theta,r}(\mathbf{x})}}$$

$$h_{\mathbf{p}_{\theta}}(\mathbf{x}) = \min(\text{argmax}_{k \in \{1, \dots, K\}} p_{\theta,k}(\mathbf{x}))$$

**Choose the most probable class, breaking ties with the smallest index.
Fit with the same cross-entropy loss.**

In this lecture...

We will build neural networks using the mathematics we already know.

Affine layers and activation functions

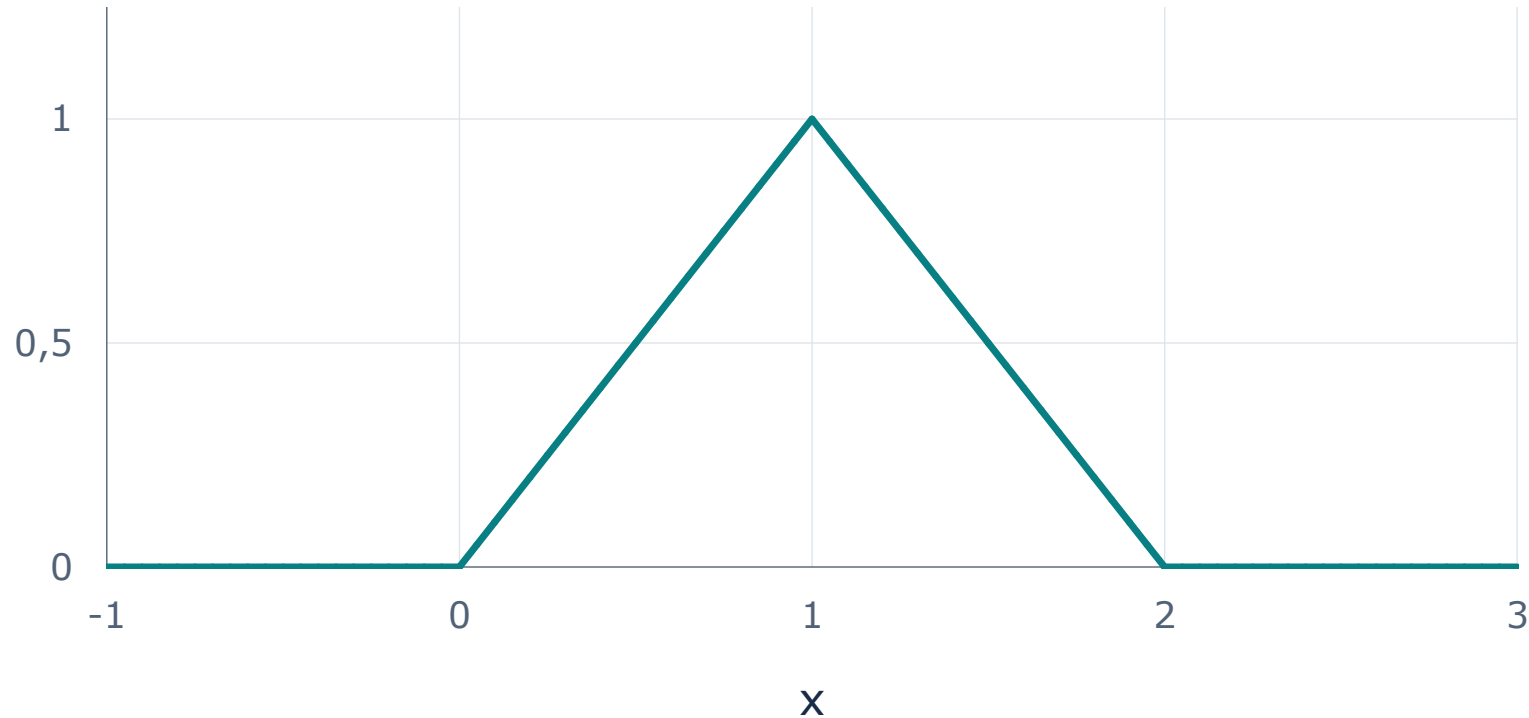
Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

A tent function

We could choose this function as a fixed feature.

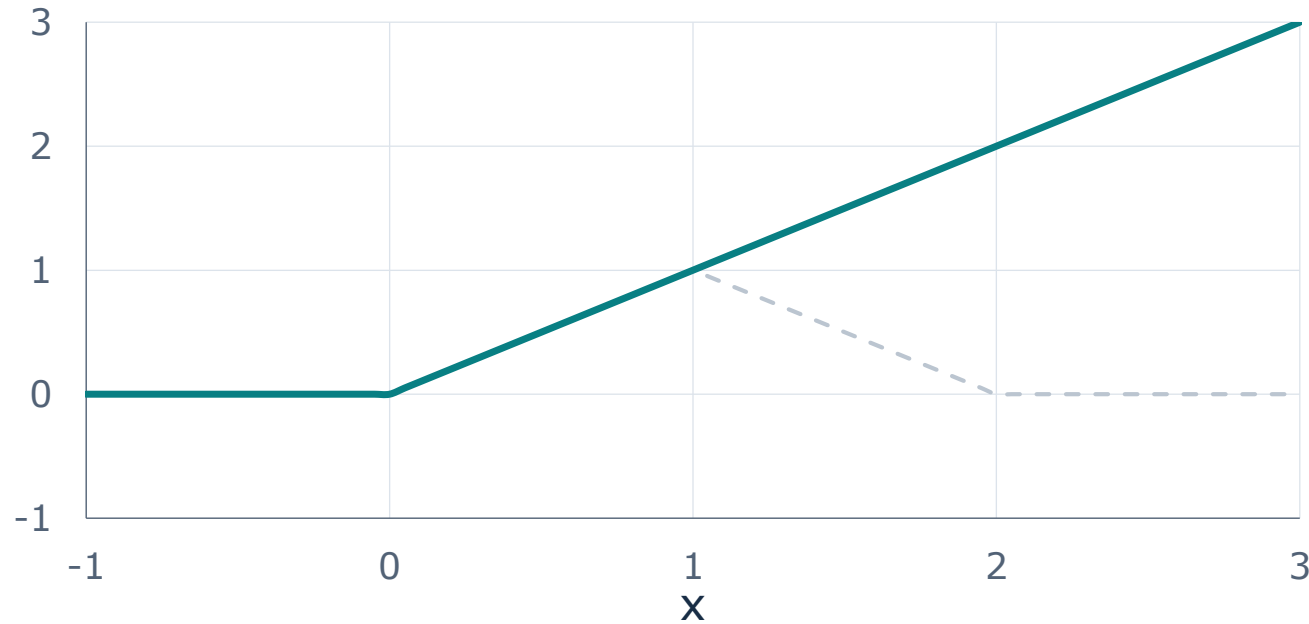


Can we build it from ReLU units?

Building a tent function: start

At $x = 0$, the slope changes from 0 to +1.

$$T_1(x) = \text{ReLU}(x)$$



One hidden unit

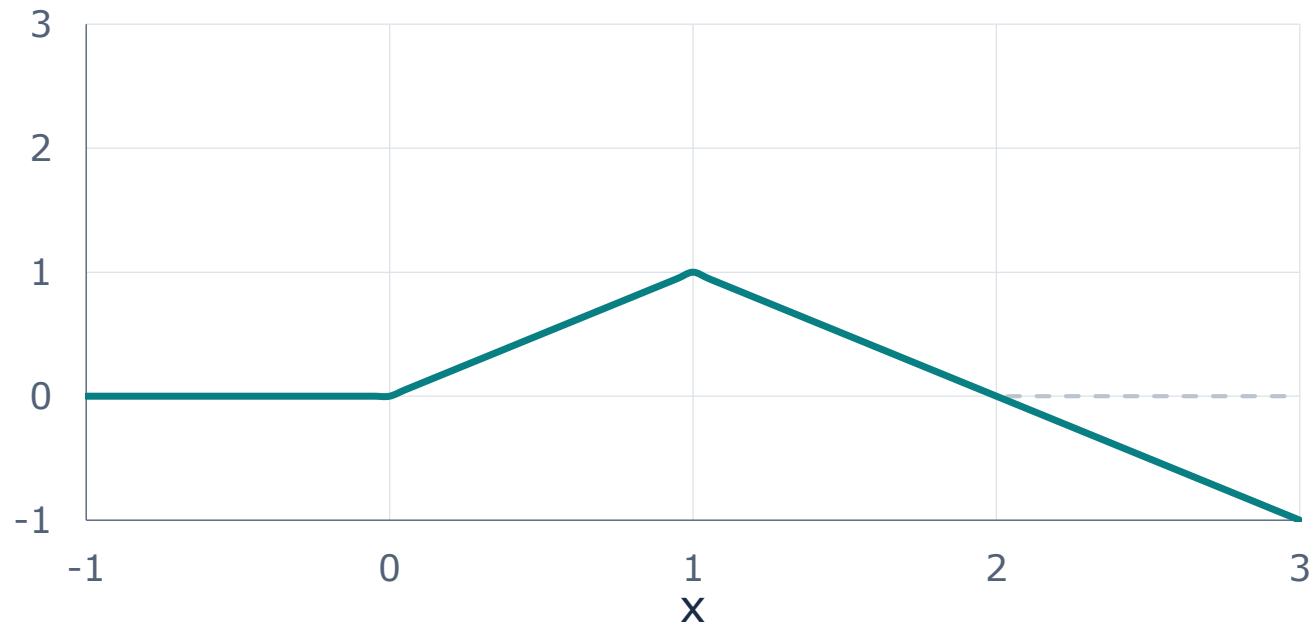
The target returns to zero.

One ReLU creates one change of slope.

Building a tent function: turn

At $x = 1$, the second unit changes the slope by -2 .

$$T_2(x) = \text{ReLU}(x) - 2\text{ReLU}(x - 1)$$



Two hidden units

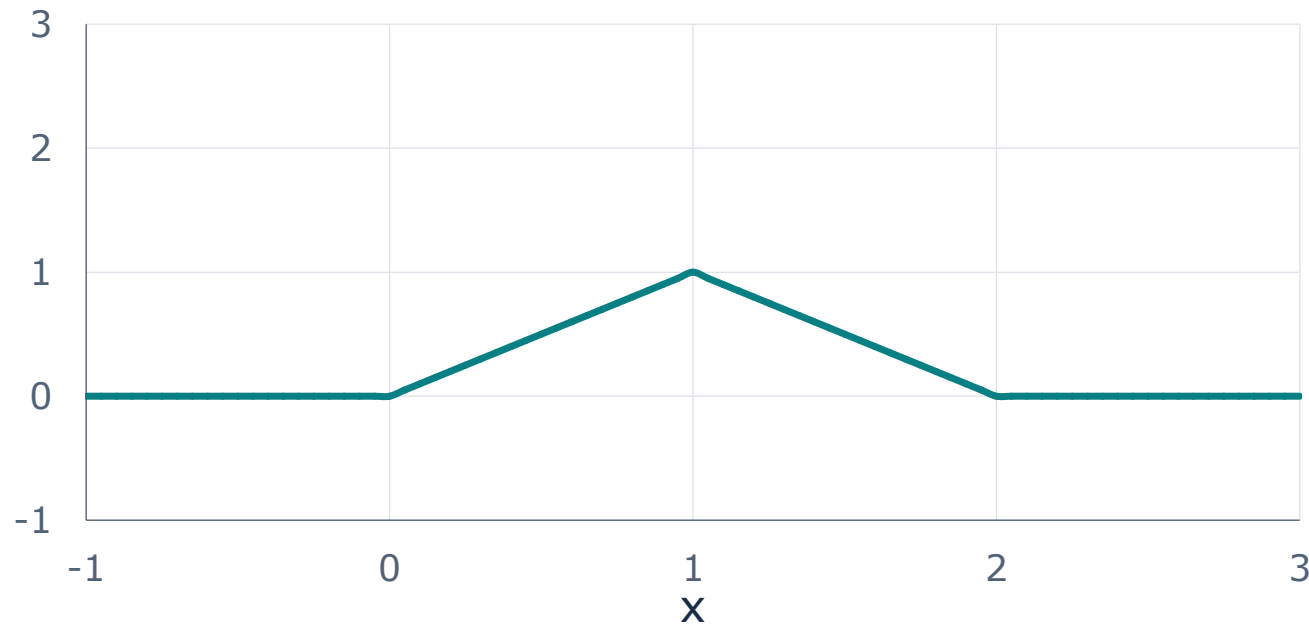
The slope is now -1 ,
but must become zero.

The output weights can be negative.

Building a tent function: stop

At $x = 2$, the third unit changes the slope by $+1$.

$$T(x) = \text{ReLU}(x) - 2\text{ReLU}(x - 1) + \text{ReLU}(x - 2)$$



Three hidden units

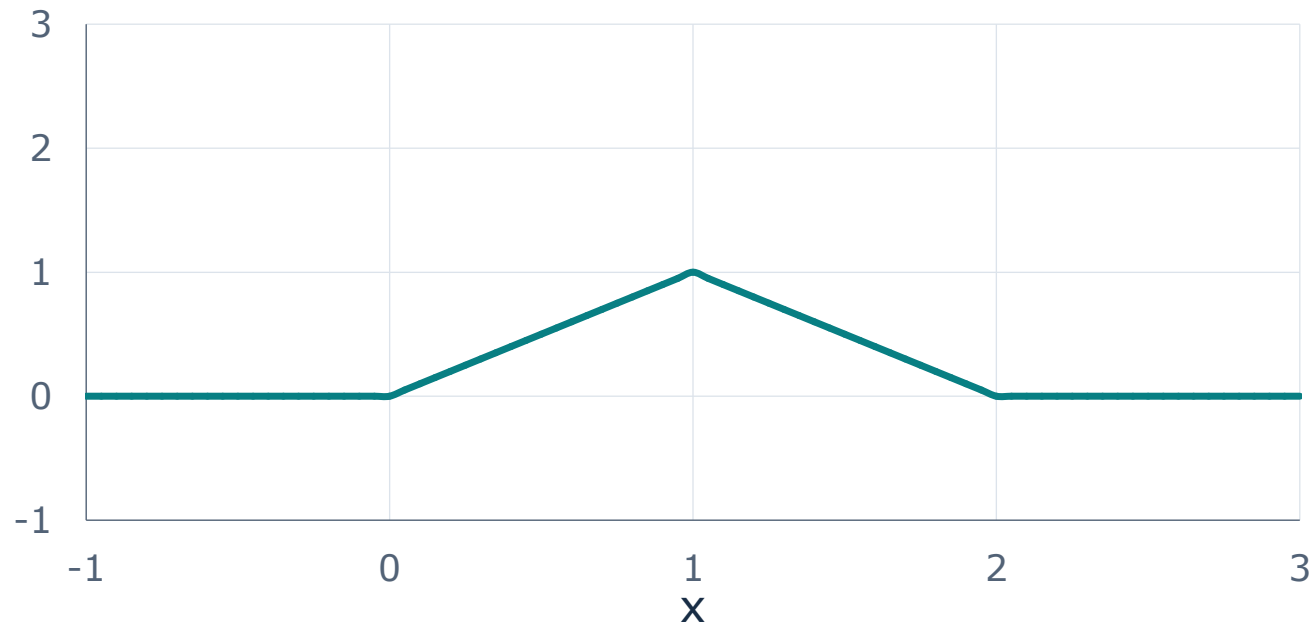
The slope returns to zero.
The tent is complete.

Three ReLU units produce the tent function.

The parameters of the tent network

One input, three hidden units and one affine output.
These parameters can be fitted to data instead of chosen by hand.

$$T(x) = \text{ReLU}(x) - 2\text{ReLU}(x - 1) + \text{ReLU}(x - 2)$$



$$\mathbf{W} = (1, 1, 1)$$

$$\mathbf{b} = (0, -1, -2)$$

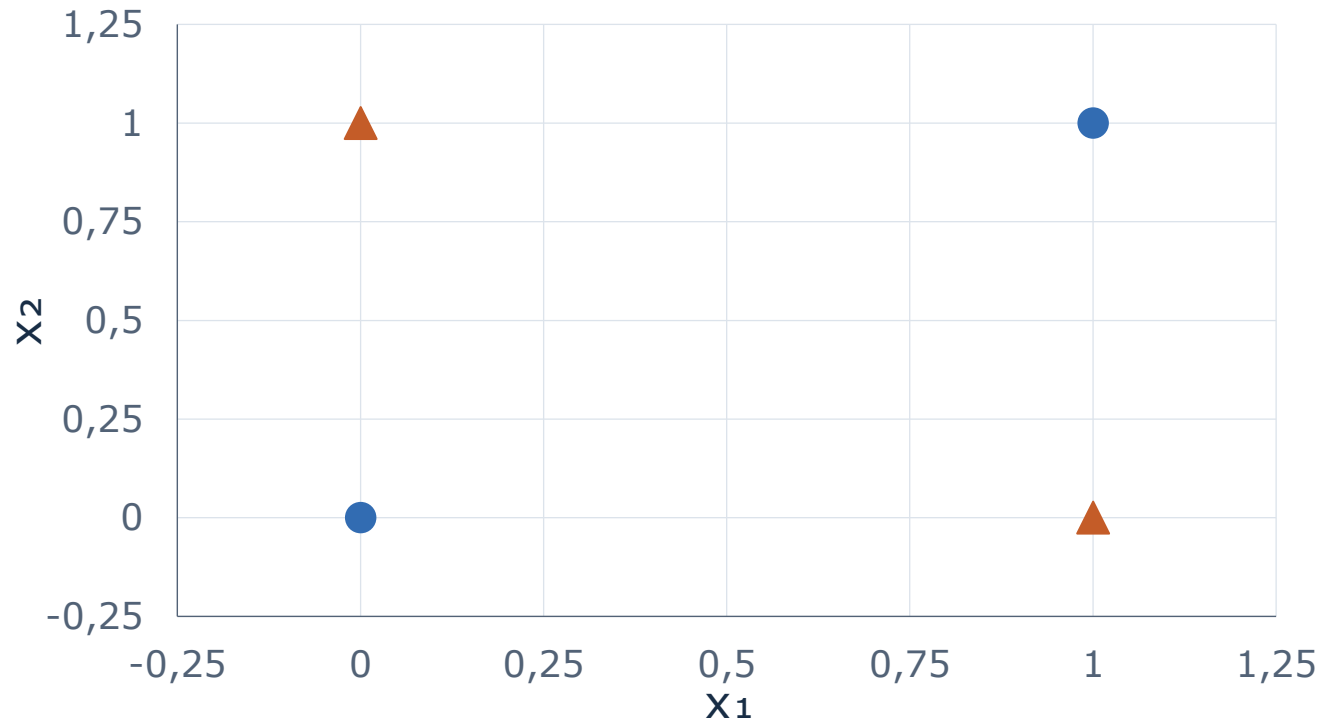
$$\mathbf{W}_{\text{out}} = (1, -2, 1)^{\top}$$

$$b_{\text{out}} = 0$$

The biases place the breakpoints. The output weights change the slopes.

The XOR problem

The label is 1 when exactly one input coordinate is 1.



$(0,0) \mapsto 0, \quad (1,1) \mapsto 0$

$(1,0) \mapsto 1, \quad (0,1) \mapsto 1$

Can one straight decision boundary classify all four observations?

Why an affine score cannot solve XOR

The score must be positive for class 1 and nonpositive for class 0.

$$s(\mathbf{x}) = b + w_1x_1 + w_2x_2$$

Class 0 observations

$$b \leq 0, \quad b + w_1 + w_2 \leq 0$$

$$2b + w_1 + w_2 \leq 0$$

Class 1 observations

$$b + w_1 > 0, \quad b + w_2 > 0$$

$$2b + w_1 + w_2 > 0$$

Adding the inequalities gives a contradiction.

Two features that solve XOR

A hidden layer can transform the observations before the affine output.

$$\phi_1(\mathbf{x}) = \text{ReLU}(x_1 - x_2), \quad \phi_2(\mathbf{x}) = \text{ReLU}(x_2 - x_1)$$

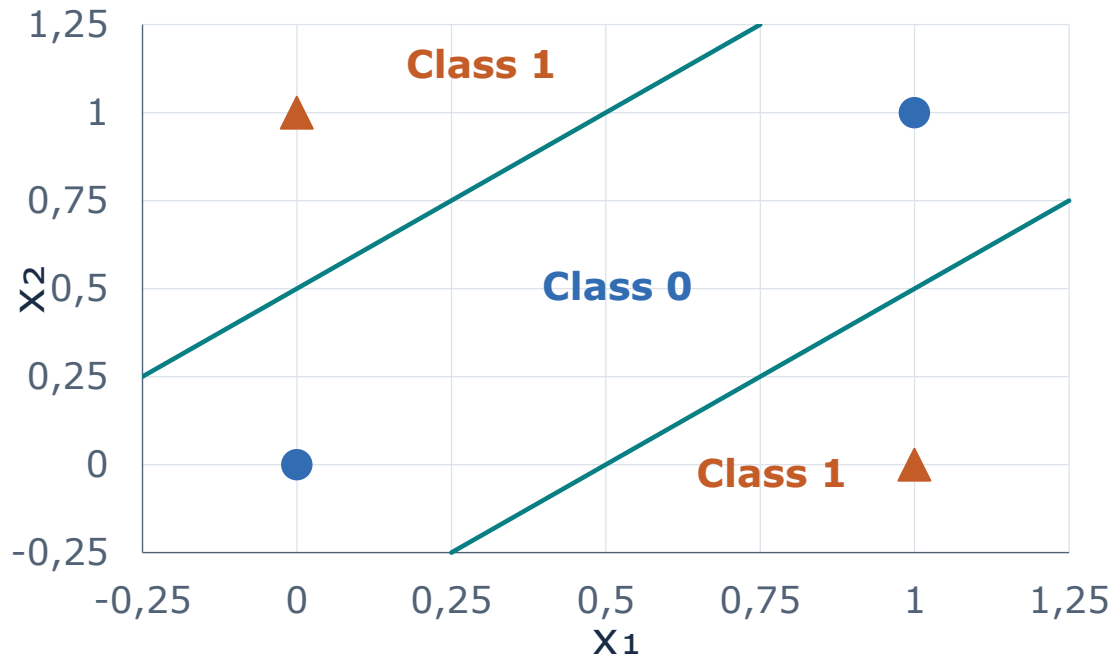
$$f(\mathbf{x}) = \phi_1(\mathbf{x}) + \phi_2(\mathbf{x}) - \frac{1}{2} = |x_1 - x_2| - \frac{1}{2}$$

Input row	Feature row	Score	Label
(0, 0)	(0, 0)	-0.5	0
(1, 0)	(1, 0)	+0.5	1
(0, 1)	(0, 1)	+0.5	1
(1, 1)	(0, 0)	-0.5	0

A straight boundary in feature space

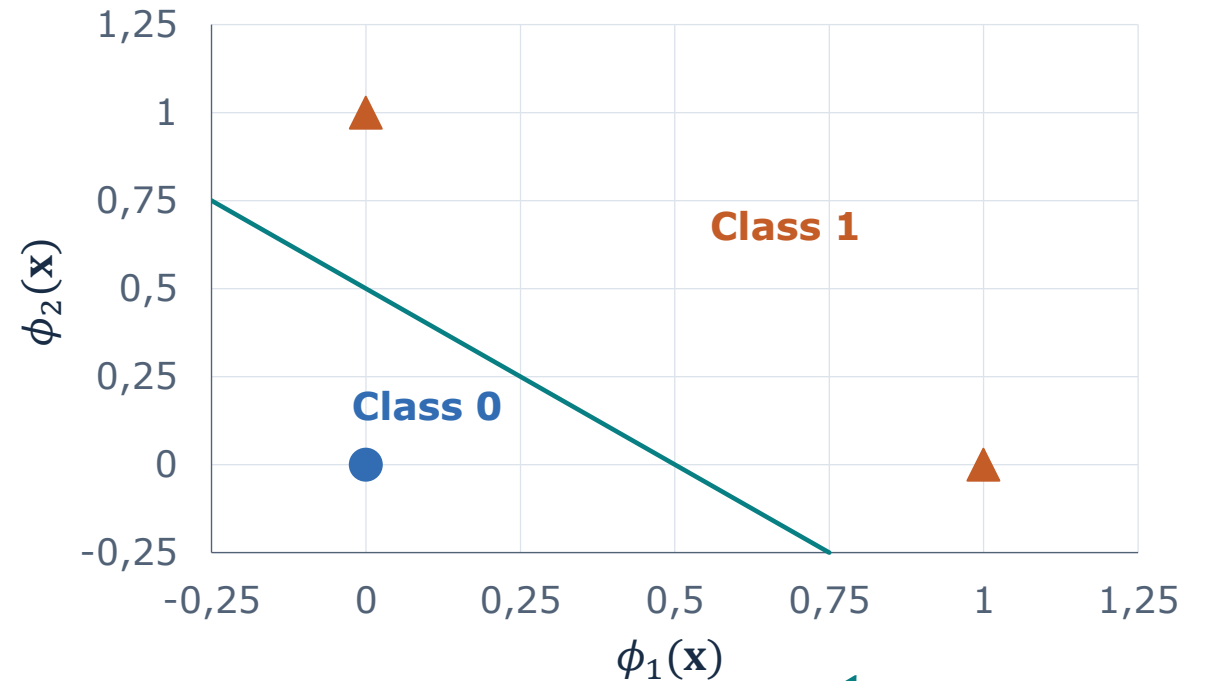
An affine output can produce a **nonlinear** decision boundary in the original input space.

Input space



$$|x_1 - x_2| = \frac{1}{2}$$

Feature space



$$\phi_1(\mathbf{x}) + \phi_2(\mathbf{x}) = \frac{1}{2}$$

(0,0) and (1,1) both map to the origin.

In this lecture...

We will build neural networks using the mathematics we already know.

Affine layers and activation functions

Neural networks and the forward pass

What neural networks can represent

Fitting neural networks and local minima

Fitting all weights and biases

The observations stay fixed. The network parameters vary.

Regression

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \sum_{i=1}^N \ell_{\text{sq}}(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i),$$

Binary Classification

$$\mathcal{L}_{\text{log}}(\boldsymbol{\theta}) = \sum_{i=1}^N \ell_{\text{log}}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i)$$

Where the loss is differentiable, we can update the network parameters by **gradient descent**.

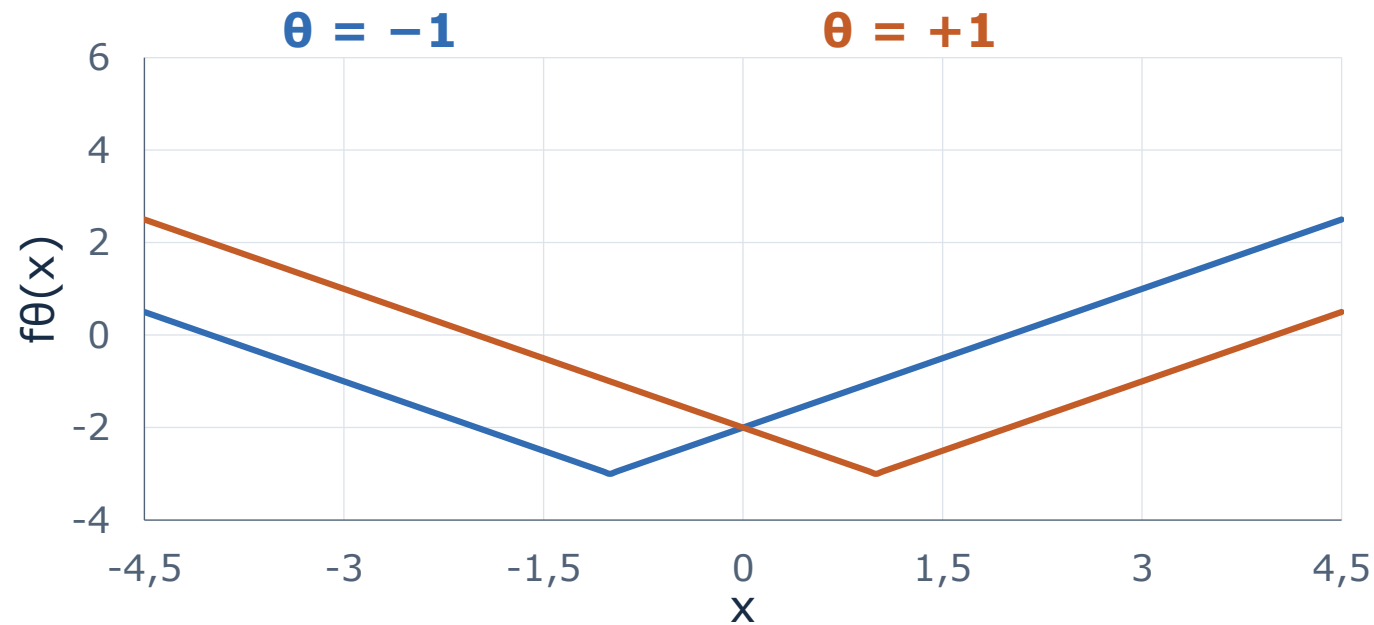
$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla \mathcal{L}(\boldsymbol{\theta}^{(t)}), \quad \eta > 0$$

**For neural networks,
will gradient descent converge to a global minimum?**

A network with one adjustable parameter

Only the scalar θ varies. All other coefficients stay fixed.

$$f_{\theta}(x) = \text{ReLU}(x - \theta) + \text{ReLU}(\theta - x) - 3 = |x - \theta| - 3$$



Hidden biases: $-\theta$ and $+\theta$

The two biases cannot change independently.

Changing θ moves where the vertex is.

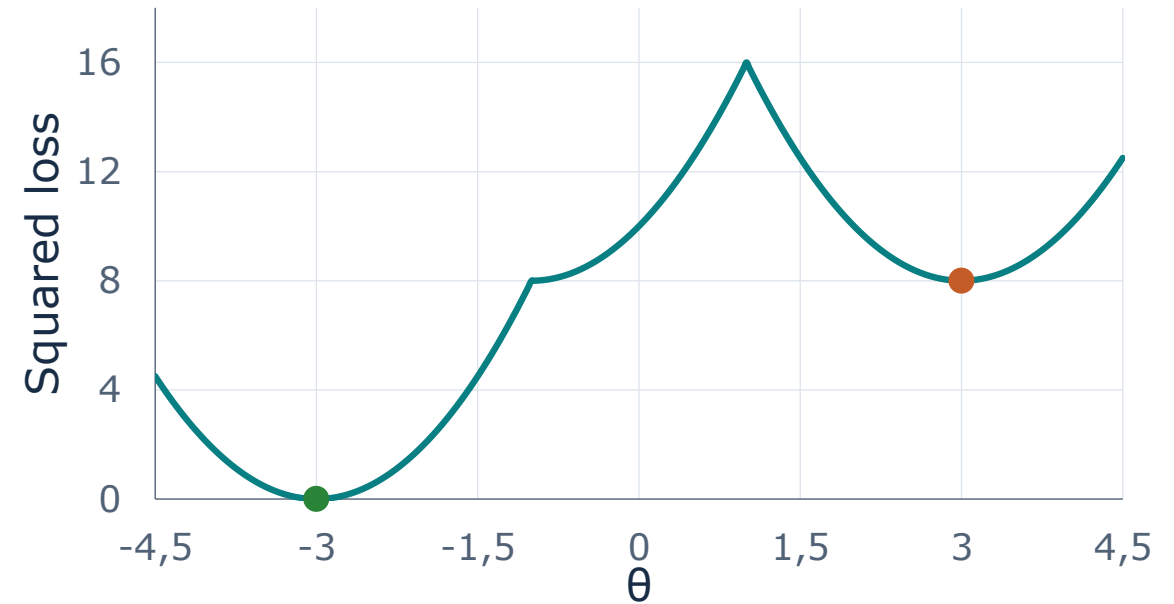
We will optimise within this one-parameter family.

The regression loss

Two observations: $(-1, -1)$ and $(1, 1)$.

$$\mathcal{L}_{\text{sq}}(\theta) = (f_{\theta}(-1) + 1)^2 + (f_{\theta}(1) - 1)^2$$

$$\mathcal{L}_{\text{sq}}(\theta) = \begin{cases} 2(\theta + 3)^2 & \theta \leq -1 \\ 2(\theta + 1)^2 + 8 & -1 < \theta < 1 \\ 2(\theta - 3)^2 + 8 & \theta \geq 1 \end{cases}$$



The loss is a function of θ . Its graph has two valleys.

Two stationary points, different losses

Both points have derivative zero, but their losses are different.

Parameter	Predictions at $x = -1, 1$	Squared loss
$\theta = -3$	$(-1, 1)$	0
$\theta = +3$	$(1, -1)$	8

Global minimum at $\theta = -3$

No parameter value
has a smaller loss.

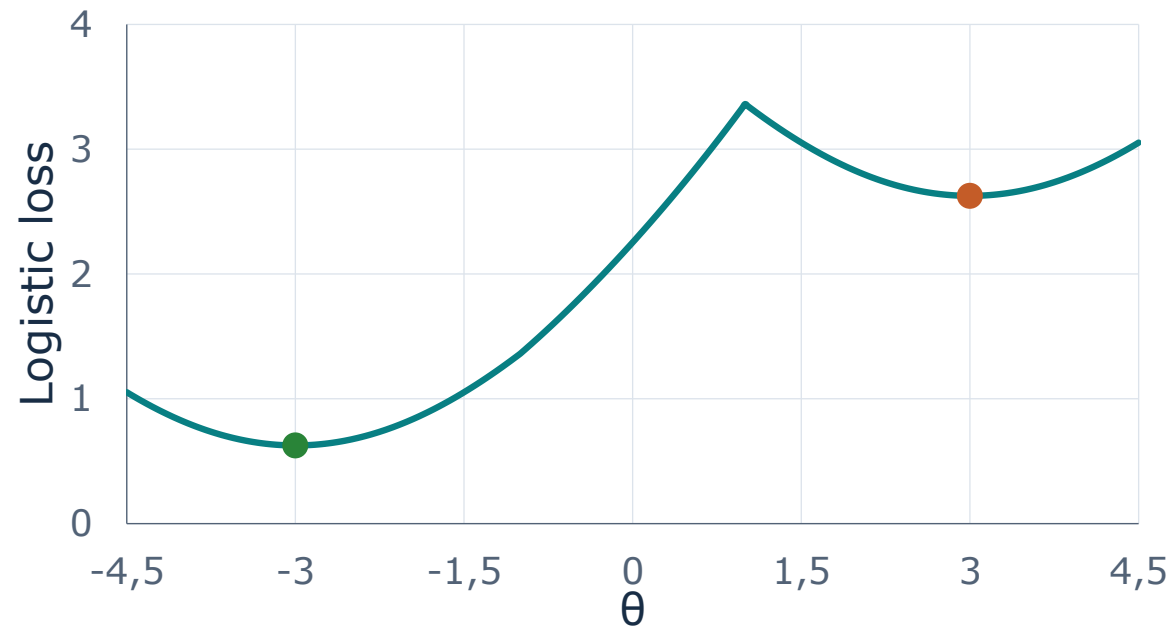
Local minimum at $\theta = +3$

No sufficiently small change
improves the loss.

The same difficulty in classification

Two observations: $(-1, 0)$ and $(1, 1)$.

$$p_{\theta}(x) = \sigma(f_{\theta}(x)), \quad \mathcal{L}_{\log}(\theta) = -\log(1 - p_{\theta}(-1)) - \log(p_{\theta}(1))$$

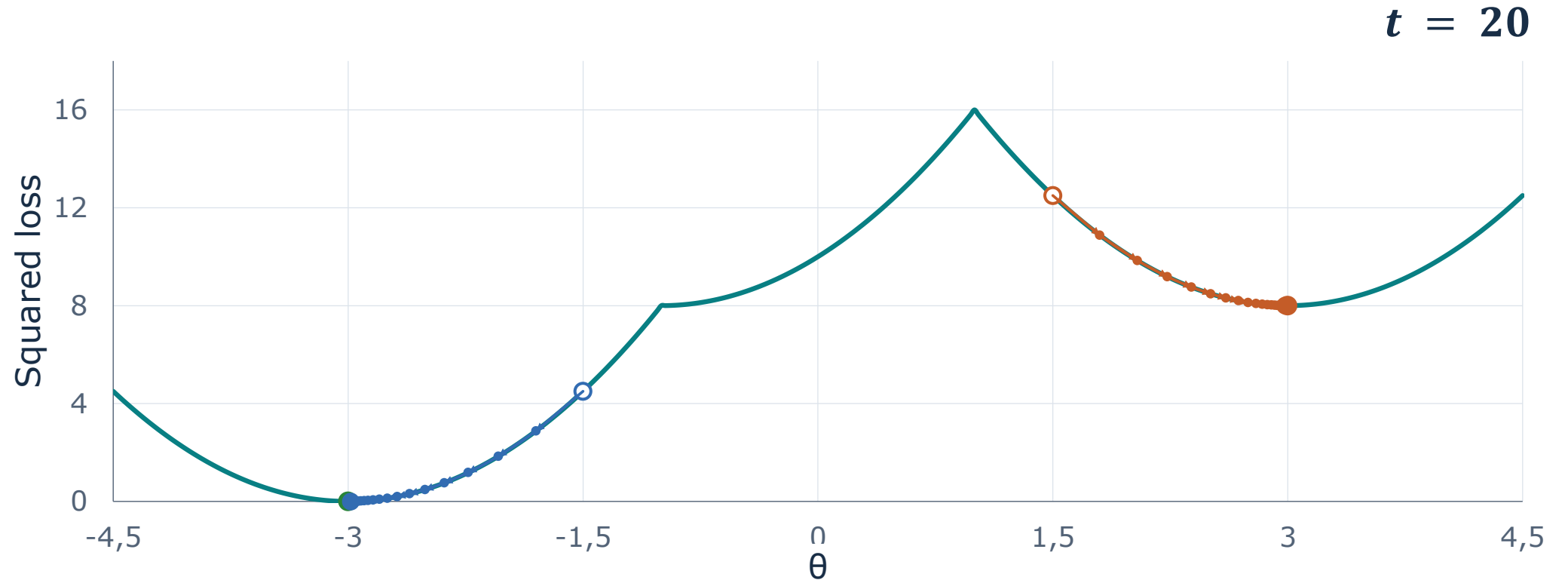


θ	Logistic loss	Mistakes
-3	0.627	0 of 2
+3	2.627	2 of 2

The local minimum can give much worse predictions than the global one.

Gradient descent: two starting points

Initial values $\theta^{(0)} = -1.5$ and $+1.5$. Learning rate $\eta = 0.05$.



Start at -1.5

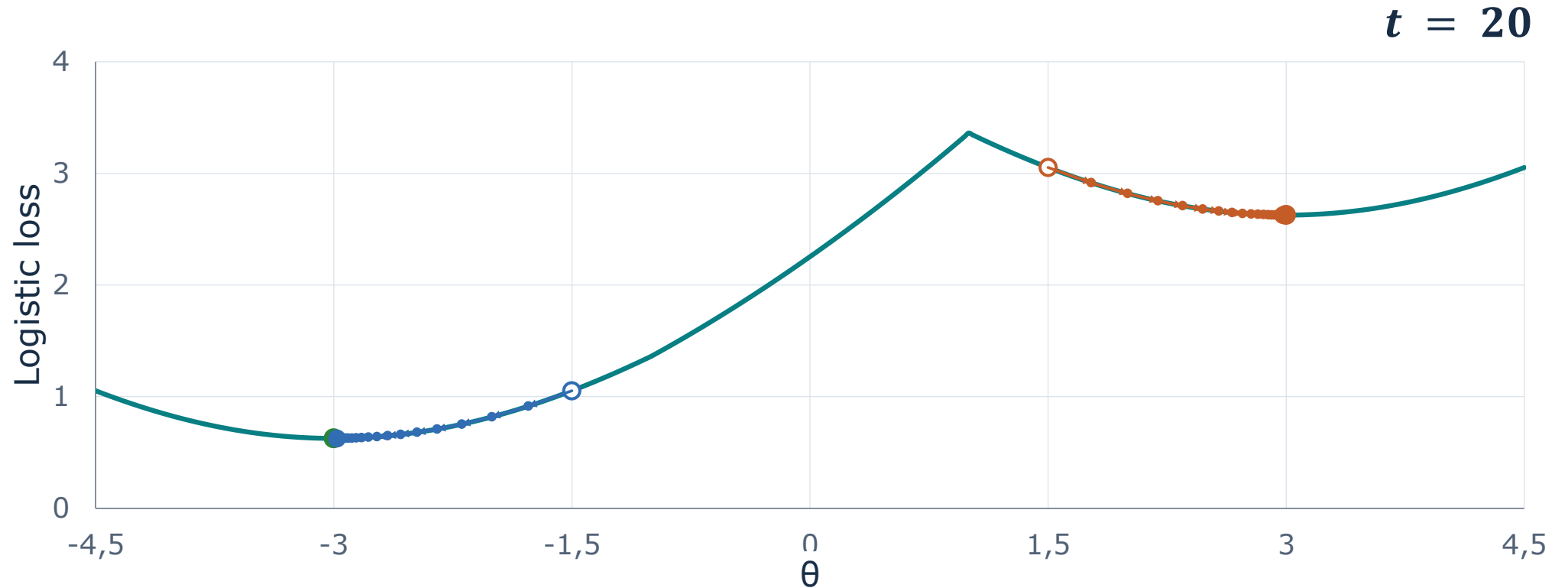
$$\theta^{(20)} \approx -2.9827, \quad \mathcal{L}(\theta^{(20)}) \approx 0.0006$$

Start at $+1.5$

$$\theta^{(20)} \approx 2.9827, \quad \mathcal{L}(\theta^{(20)}) \approx 8.0006$$

Gradient descent with logistic loss

Initial values $\theta^{(0)} = -1.5$ and $+1.5$. Learning rate $\eta = 0.5$.



Start at -1.5

$$\theta^{(20)} \approx -2.9802, \quad \mathcal{L}(\theta^{(20)}) \approx 0.6266$$

Start at $+1.5$

$$\theta^{(20)} \approx 2.9802, \quad \mathcal{L}(\theta^{(20)}) \approx 2.6266$$

Finding useful solutions in practice

These practical **heuristics** can help,
without guaranteeing convergence to a global minimum.

Try different initialisations

Different starting points can lead to different final fits.
Keep track of which runs fit the training data better.

Change how the updates are computed

Minibatches use gradients from small random subsets.
This may help the iterates leave the region around a local minimum.

Consider increasing the width

A wider network can sometimes be easier to optimise.
Increasing width **does not guarantee** that gradient descent
reaches a global minimum.

Computing the gradient

Gradient descent requires the derivative of the loss with respect to every weight and bias.

The network composes affine transformations and activation functions.
We then evaluate the loss.

Where these operations are differentiable,
we apply the chain rule through this composition.

ReLU is not differentiable at zero.

For parameter updates, we use 0 as its multiplier there.
This is a computational convention, not an ordinary derivative.

Backpropagation organises this calculation efficiently.

Next lecture: backpropagation.

Lecture notes and exercises

The notes for today's lecture are available on the course GitHub page.

They explain the material in more detail and include exercises.

Each week, I will post the exercise solutions before the next lecture.

github.com/semtem0051/2026_27



Scan for course materials